

Shiv Nadar University Chennai

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

Additional Task Study of Training Perceptrons for Logic Gates

1. Objective

The objectives of this experiment are:

- Code the perceptron learning algorithm for OR, NOT, and AND gates using Python. Show the weights after each update. Plot the decision boundary after each weight update using Python built-in packages.
- Code the perceptron learning algorithm for the XOR gate using Python. Show the weights after each update. Plot the decision boundary after each weight update using Python's built-in packages. Analyze and identify the pattern that prevents the algorithm from converging.
-
- Train an MLP for the XOR gate and plot the training loss and decision boundary. Mention at which epoch convergence occurs.

1. Perceptron Learning Algorithm for OR, AND and NOT Gates

1.1 Methodology

The perceptron learning algorithm initializes the weights and bias to zero and updates them whenever an incorrect prediction is made. The weight and bias updates are given by:

$$w_{\text{new}} = w_{\text{old}} + \eta \times \text{error} \times x$$

$$b_{\text{new}} = b_{\text{old}} + \eta \times \text{error}$$

where η is the learning rate and

$$\text{error} = \text{target} - \text{prediction}.$$

The learning rate used in the implementation is 1.

1.2 OR Gate

Truth Table:

x_1	x_2	Output
0	0	0
0	1	1
1	0	1
1	1	1

Weights After Each Update:

Update	w_1	w_2	Bias
1	0	0	-1
2	0	1	0
3	0	1	-1
4	1	1	0
5	1	1	-1

Decision Boundary Plots:

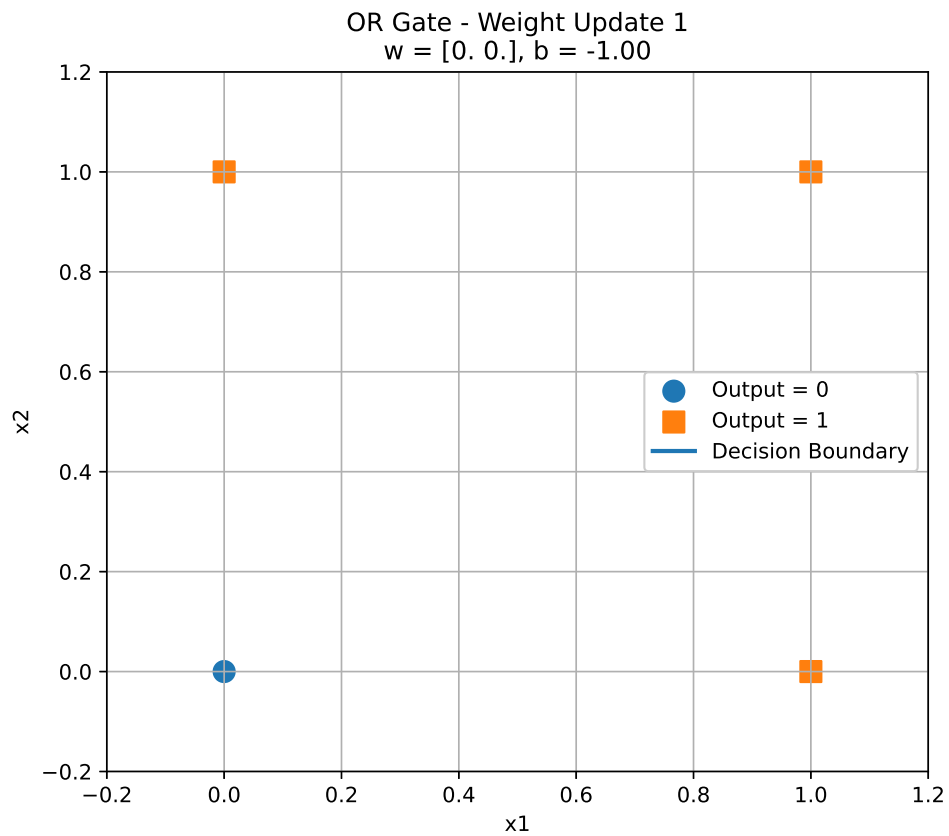


Figure 1: OR gate decision boundary after weight update 1.

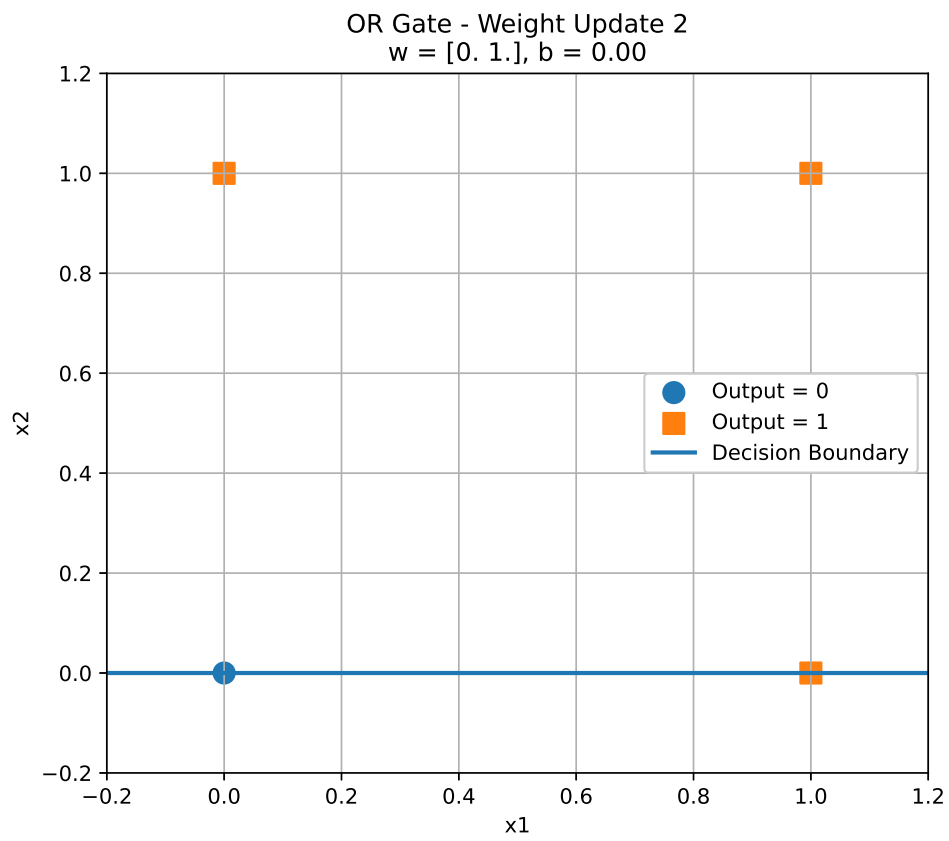


Figure 2: OR gate decision boundary after weight update 2.

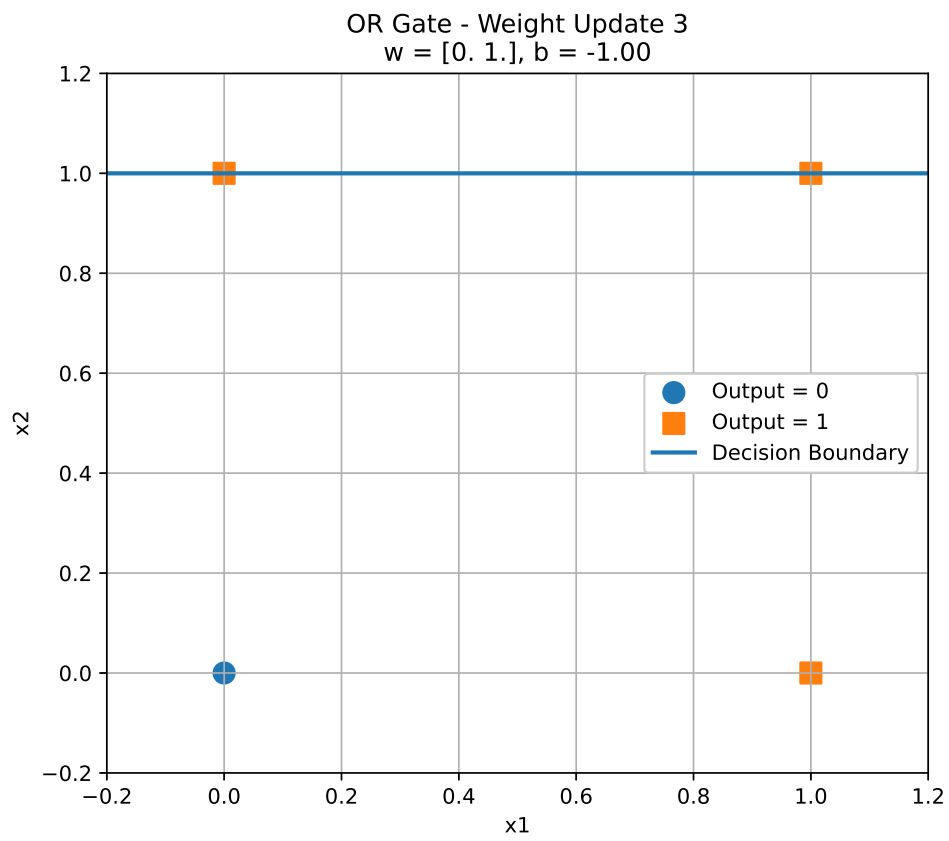


Figure 3: OR gate decision boundary after weight update 3.

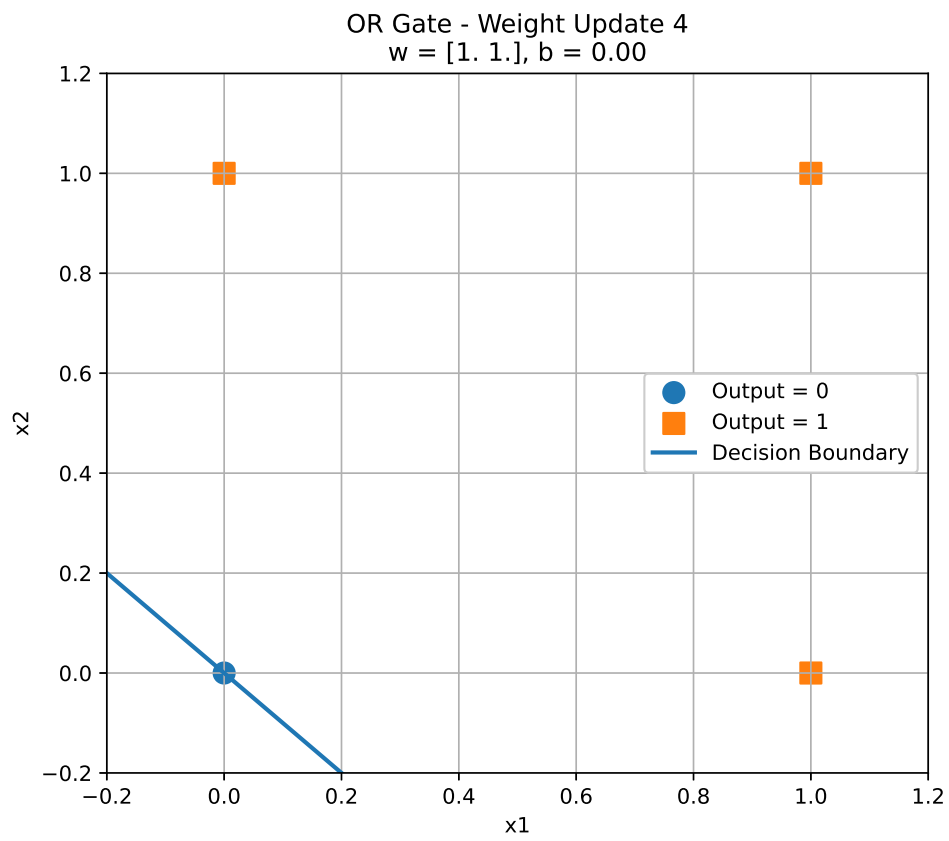


Figure 4: OR gate decision boundary after weight update 4.

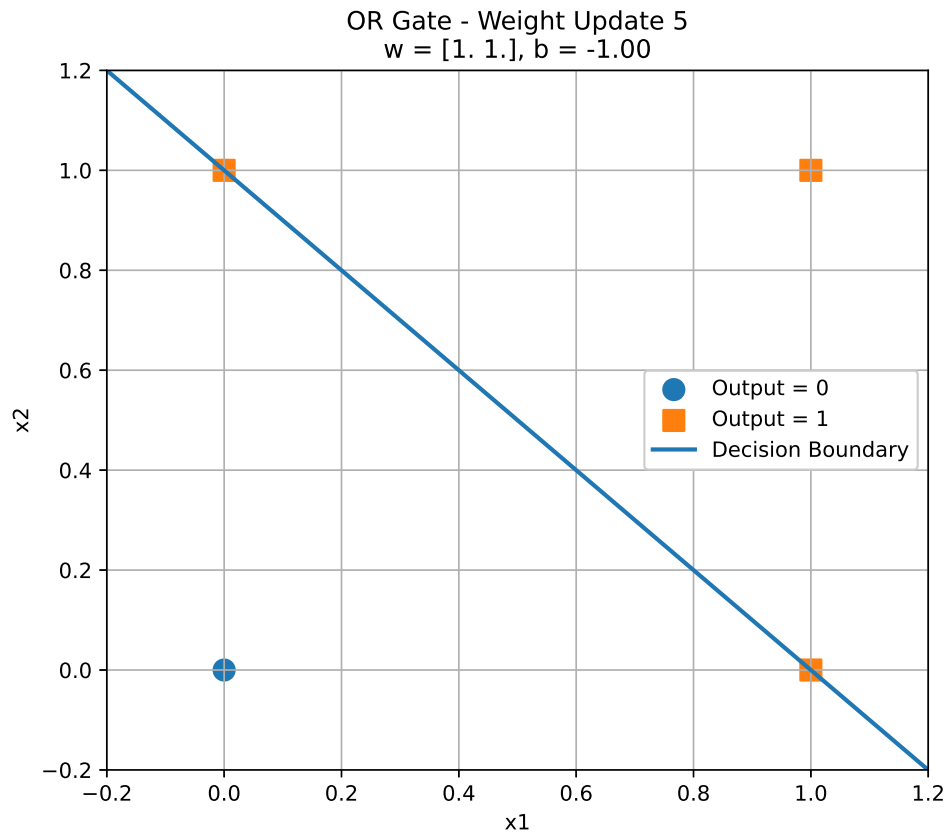


Figure 5: OR gate decision boundary after weight update 5.

Convergence:

The OR perceptron converged after **4 epochs**, with final weights $[1, 1]$ and final bias -1 .

Observation:

The perceptron continuously changes the values of the weights and the bias to successfully categorize the OR gate inputs. The perceptron converged in 4 epochs, indicating that OR is linearly separable.

1.3 AND Gate

Truth Table:

x_1	x_2	Output
0	0	0
0	1	0
1	0	0
1	1	1

Weights After Each Update:

Update	w_1	w_2	Bias
1	0	0	-1
2	1	1	0
3	1	1	-1
4	1	0	-2
5	2	1	-1
6	2	0	-2
7	1	0	-3
8	2	1	-2
9	1	1	-3
10	2	2	-2
11	2	1	-3

Decision Boundary Plots:

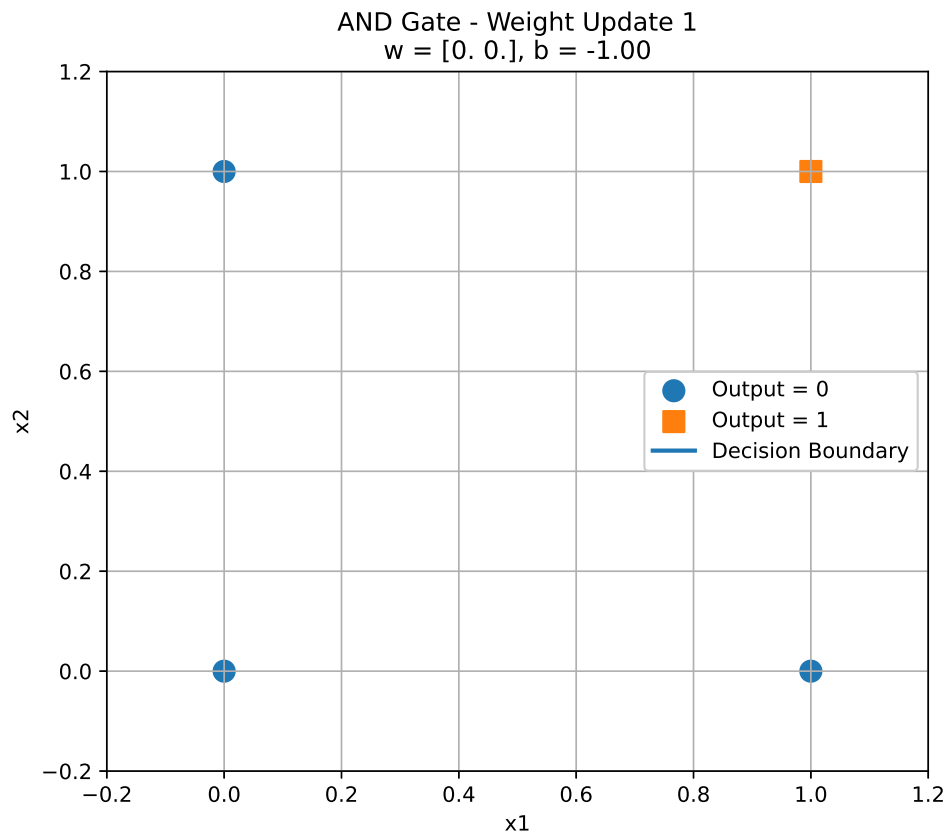


Figure 6: AND gate decision boundary after weight update 1.

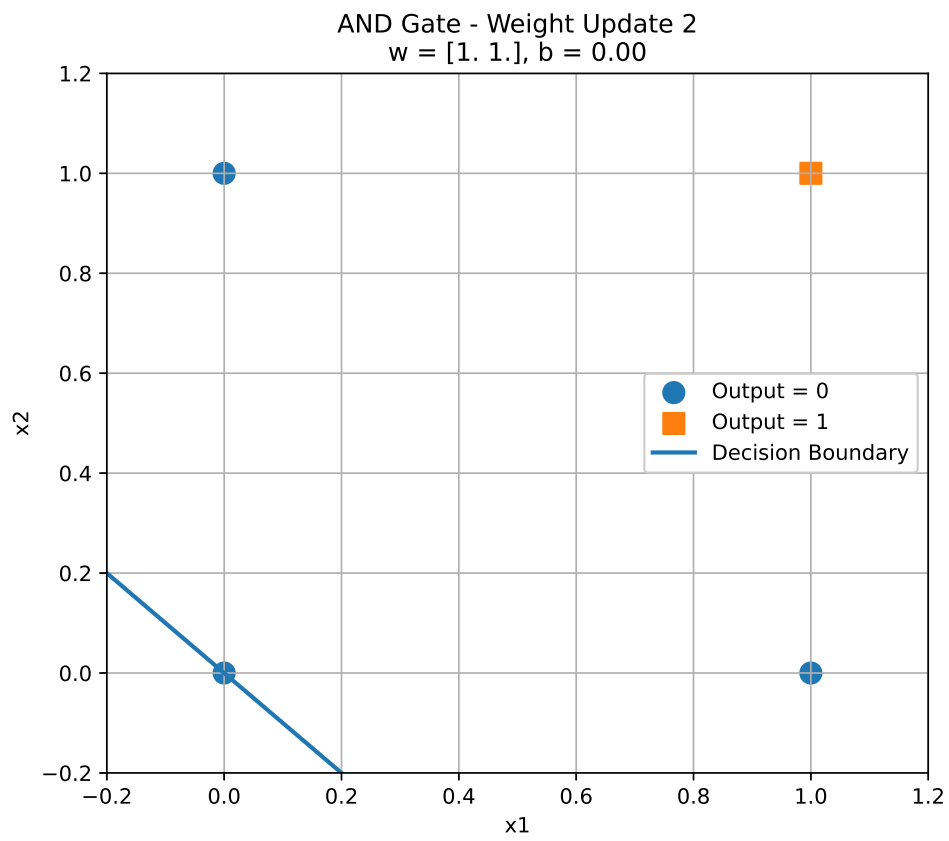


Figure 7: AND gate decision boundary after weight update 2.

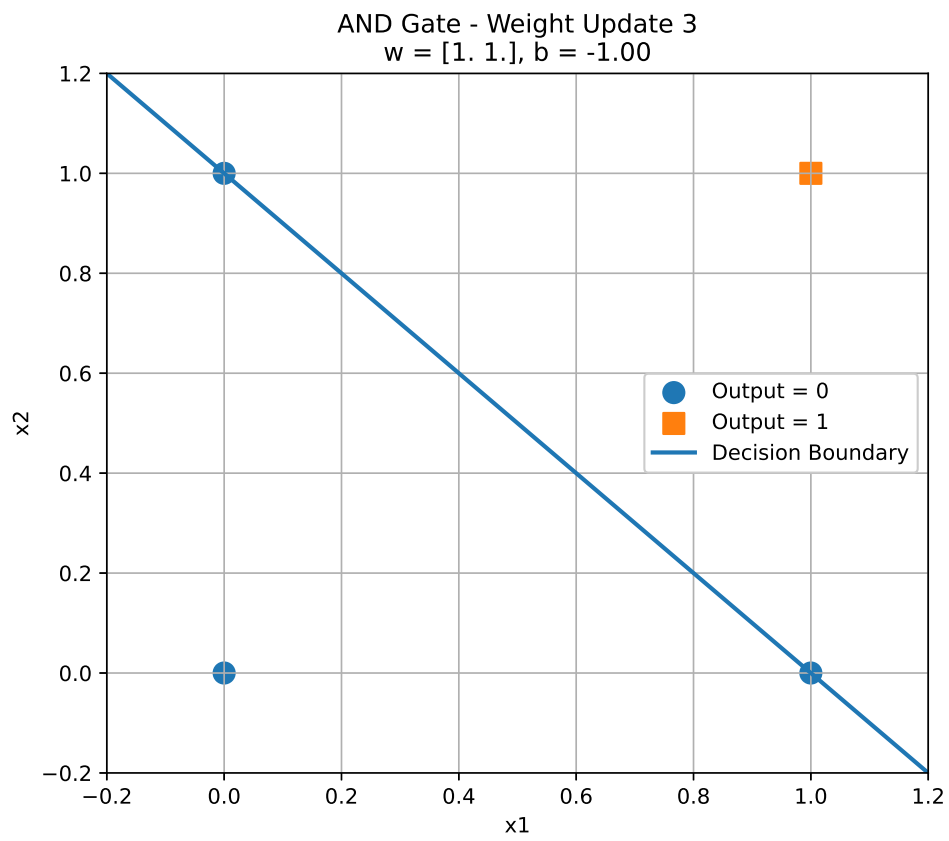


Figure 8: AND gate decision boundary after weight update 3.

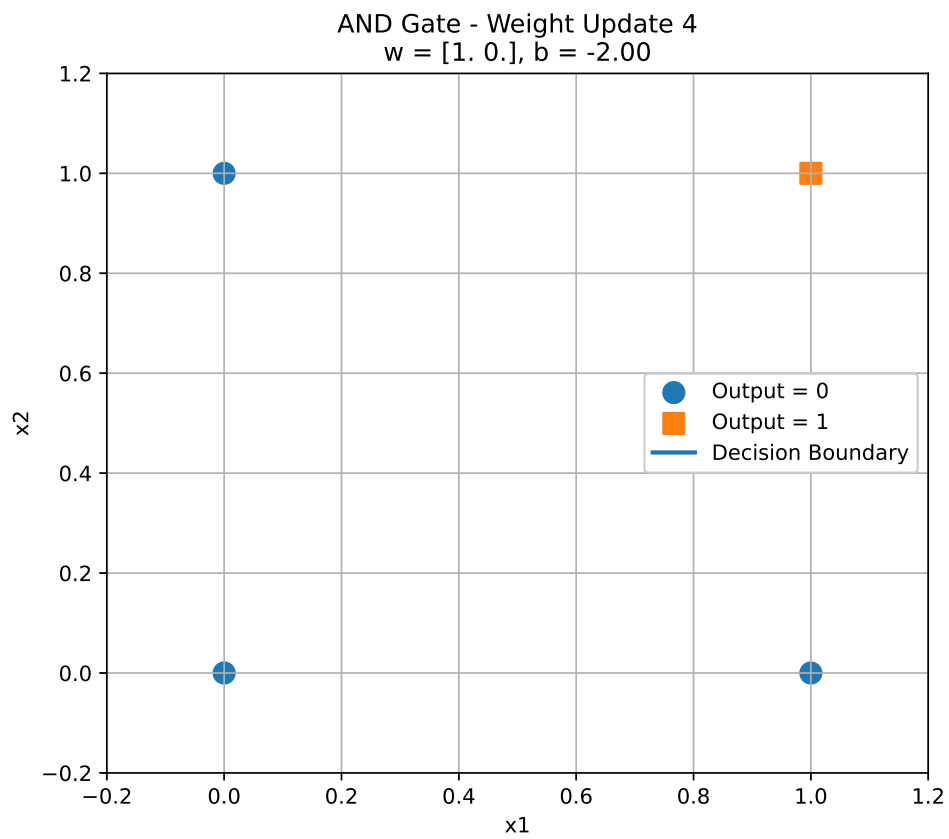


Figure 9: AND gate decision boundary after weight update 4.

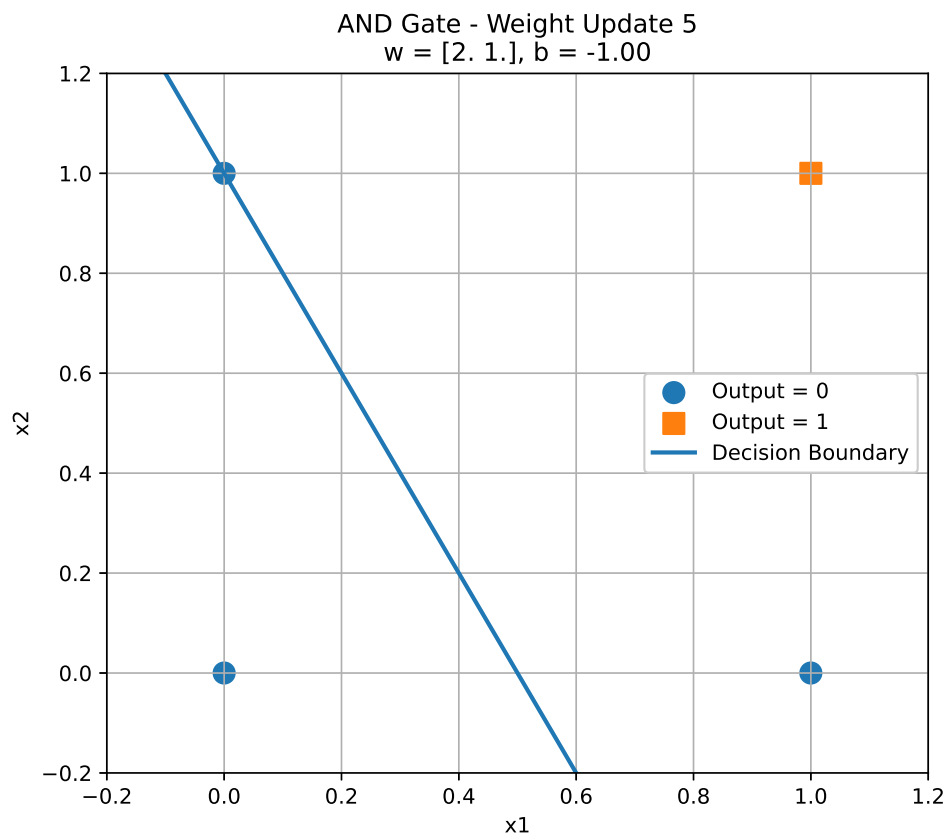


Figure 10: AND gate decision boundary after weight update 5.

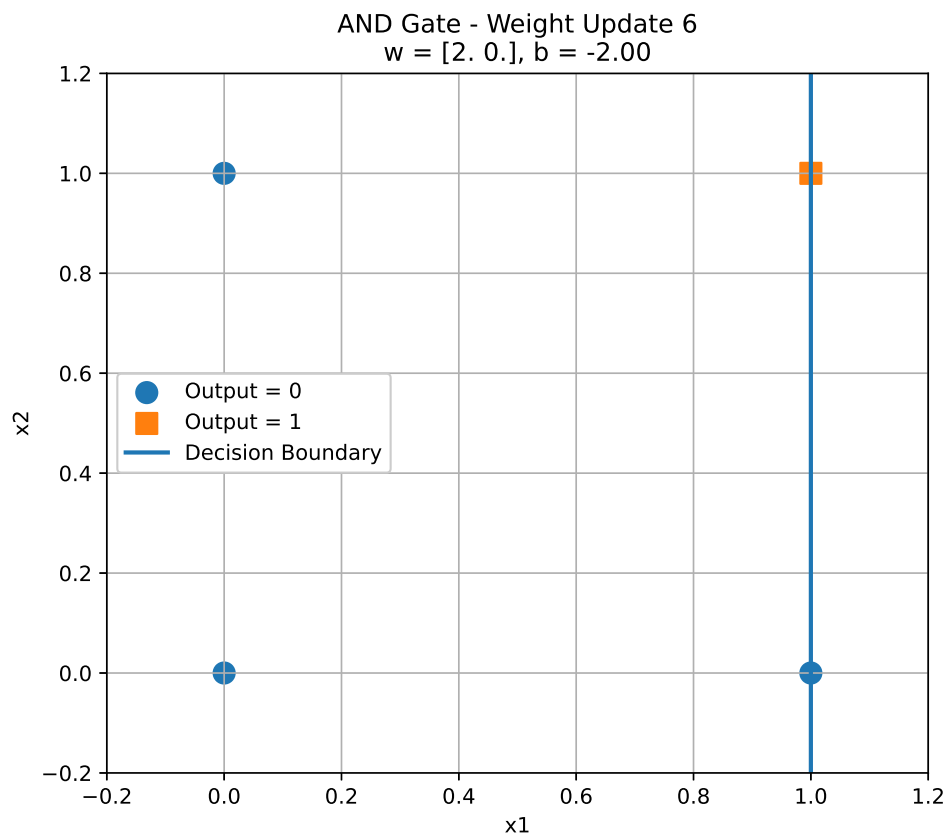


Figure 11: AND gate decision boundary after weight update 6.

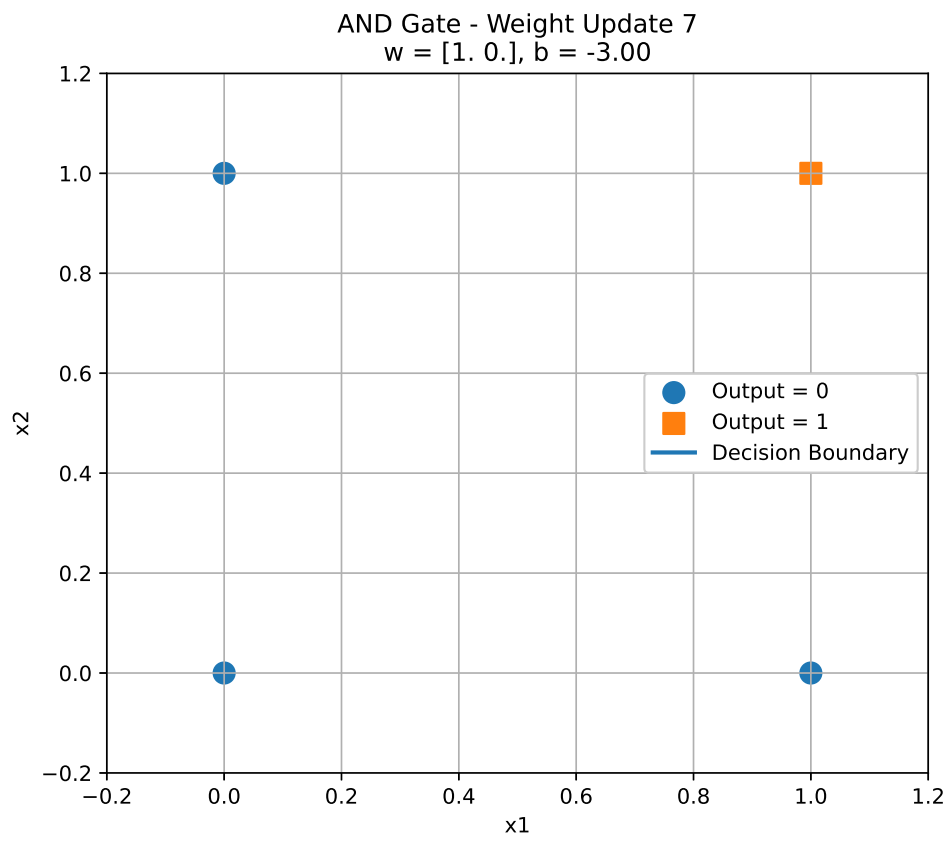


Figure 12: AND gate decision boundary after weight update 7.

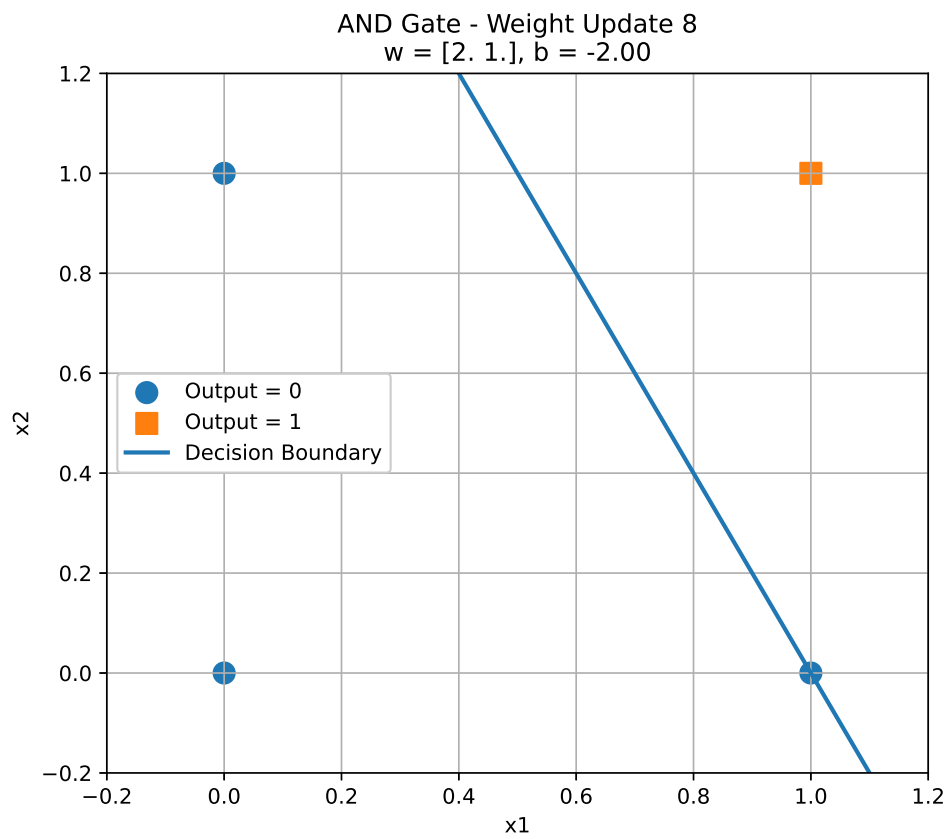


Figure 13: AND gate decision boundary after weight update 8.

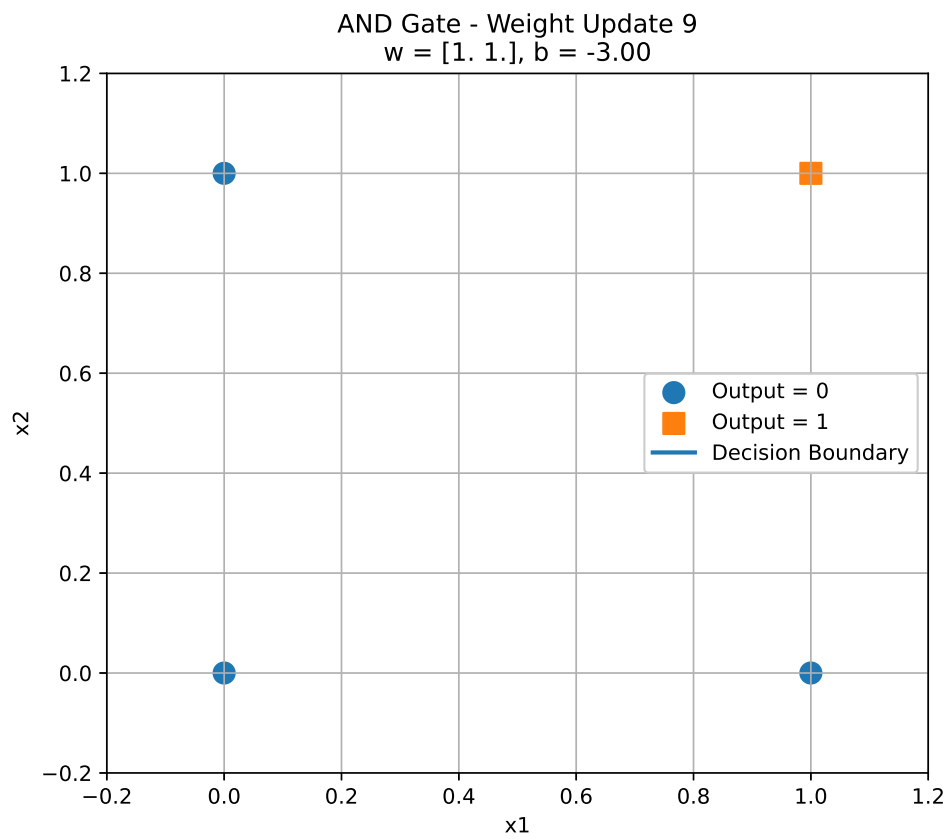


Figure 14: AND gate decision boundary after weight update 9.

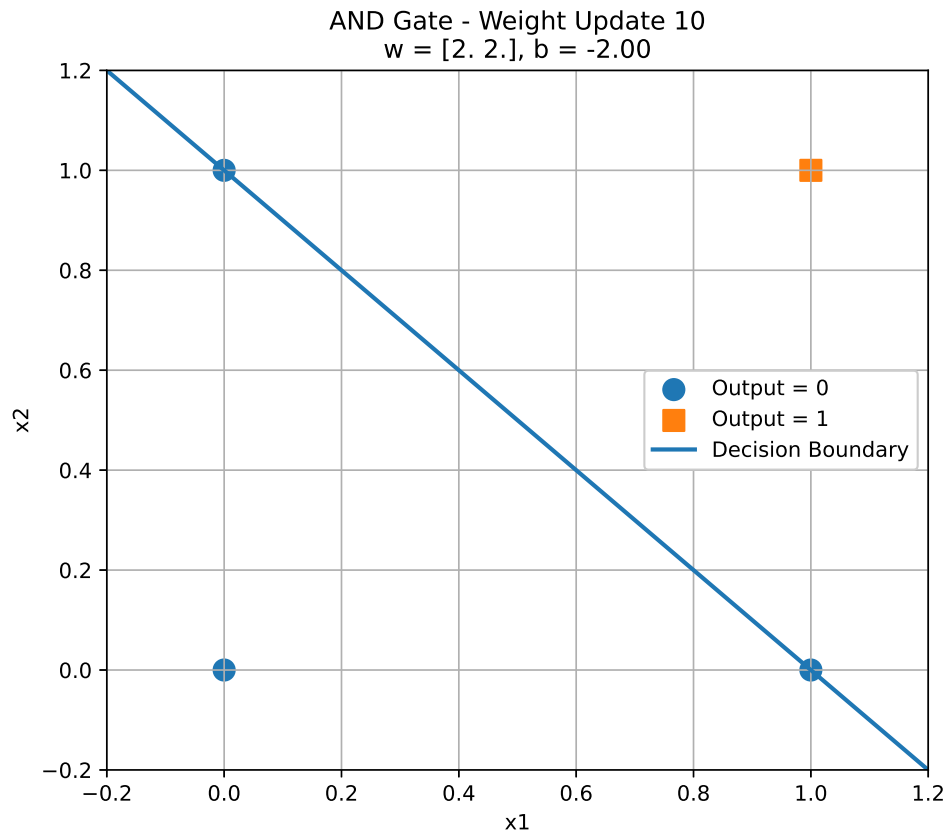


Figure 15: AND gate decision boundary after weight update 10.

Convergence:

The AND perceptron converged after **6 epochs**, with final weights $[2, 1]$ and final bias -3 .

Observation:

This procedure continues till all the samples of AND gate are classified properly. The perceptron took 6 iterations to converge and thus, proved that AND gate is linearly separable.

1.4 NOT Gate

Truth Table:

x	Output
0	1
1	0

Weights After Each Update:

Update	w_1	Bias
1	-1	-1
2	-1	0

Decision Boundary Plots:

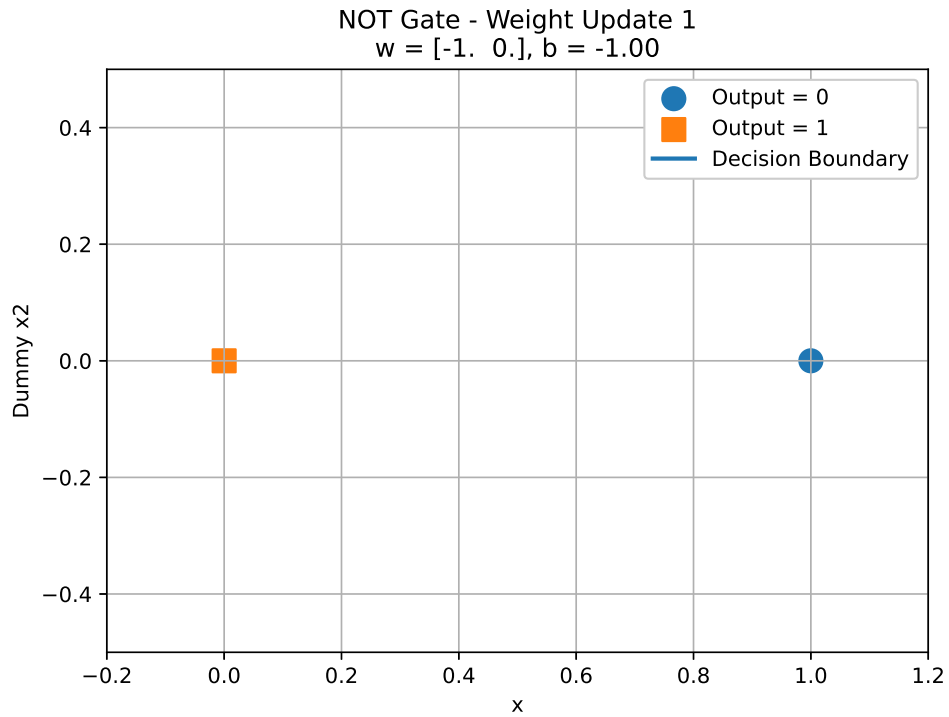


Figure 16: NOT gate decision boundary after weight update 1.

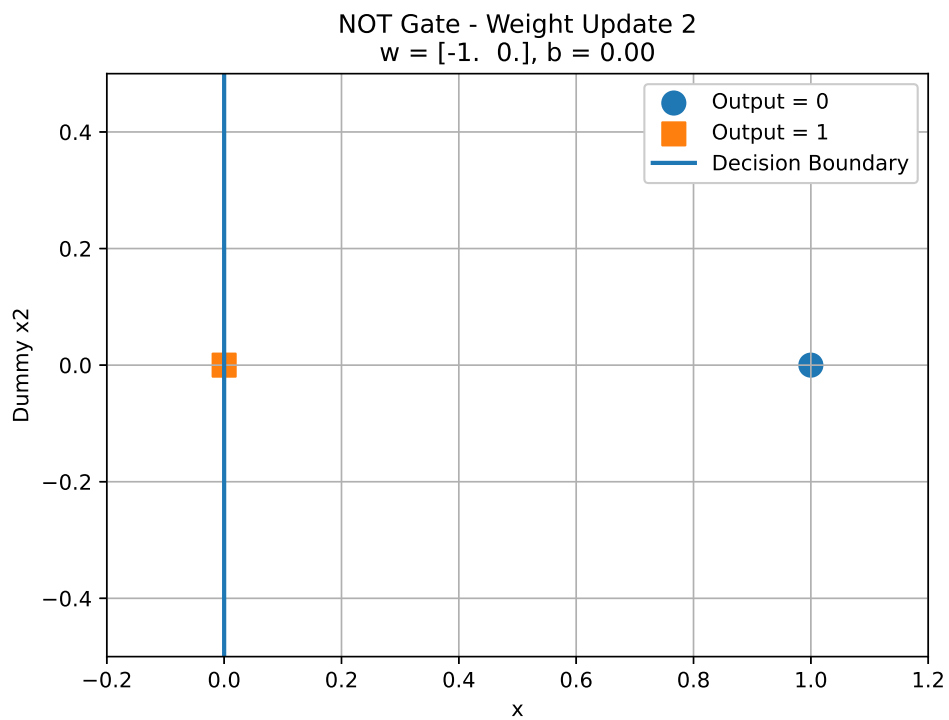


Figure 17: NOT gate decision boundary after weight update 2.

Convergence:

The NOT perceptron converged after **3 epochs**, with final weight $[-1, 0]$ and final bias 0.

Observation:

After the first misclassification of the samples, the perceptron updates the weights and bias to reach the final classification of NOT gate very soon. Convergence took place after 3 epochs, and shows that NOT function is

linearly separable.

2. Perceptron Learning Algorithm for XOR Gate

Truth Table:

x_1	x_2	Output
0	0	0
0	1	1
1	0	1
1	1	0

Parameters:

Learning Rate	1
Maximum Updates	10
Initial Weights	[0,0]
Initial Bias	0

Weights After Each Update:

Update	w_1	w_2	Bias
1	0	0	-1
2	0	1	0
3	-1	0	-1
4	-1	1	0
5	0	1	1
6	-1	0	0
7	-1	0	-1
8	-1	1	0
9	0	1	1
10	-1	0	0

Decision Boundary Plots:

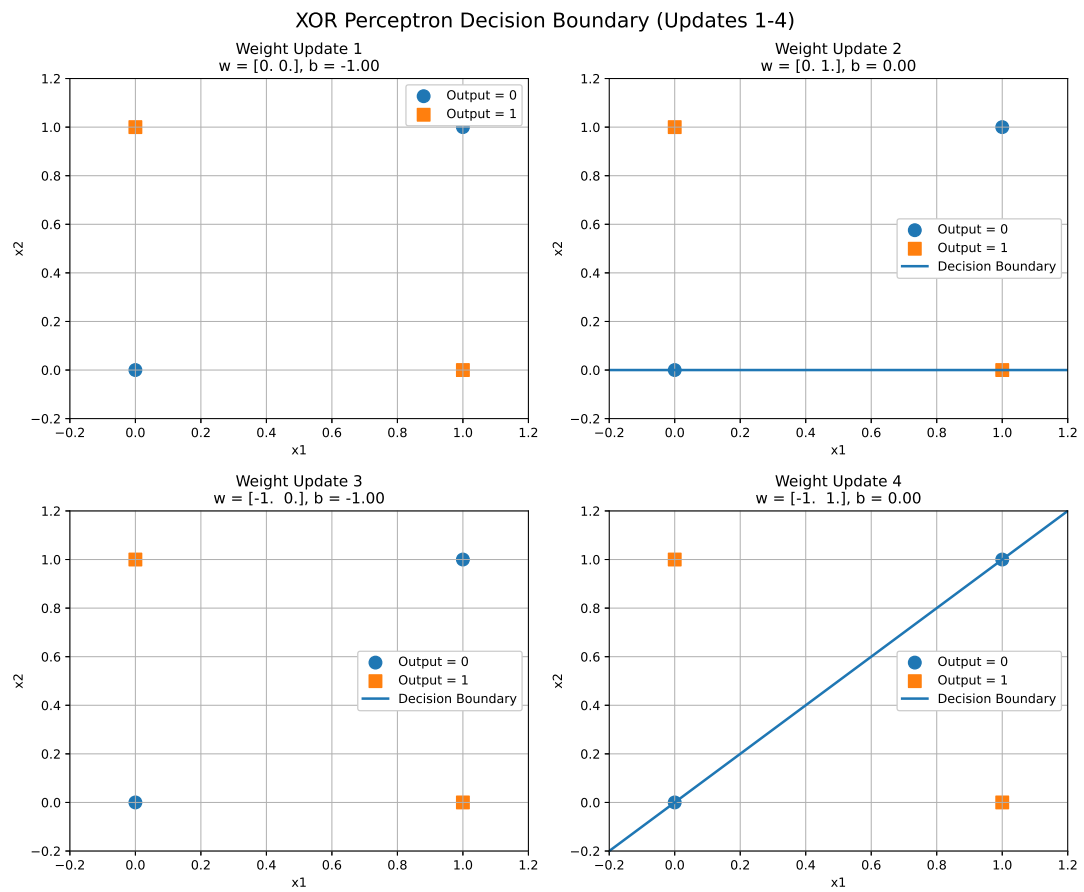


Figure 18: XOR perceptron decision boundaries for weight updates 1–4.

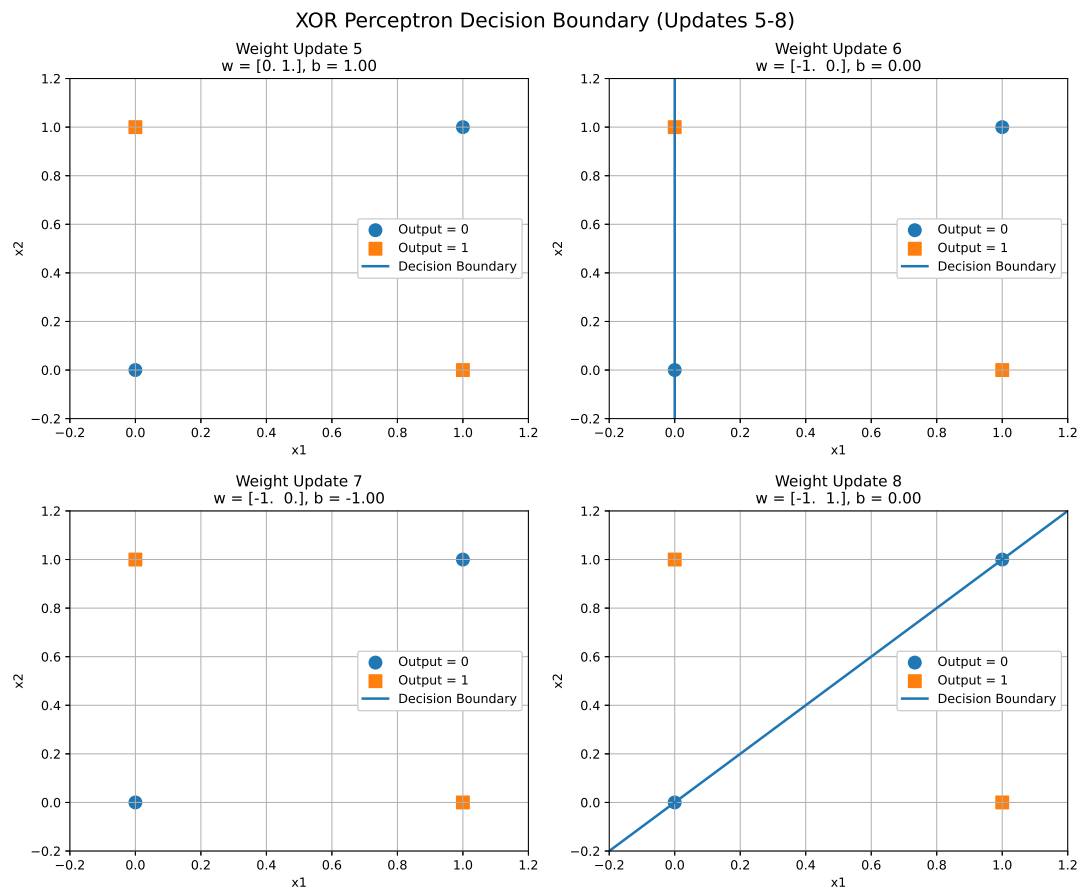


Figure 19: XOR perceptron decision boundaries for weight updates 5–8.

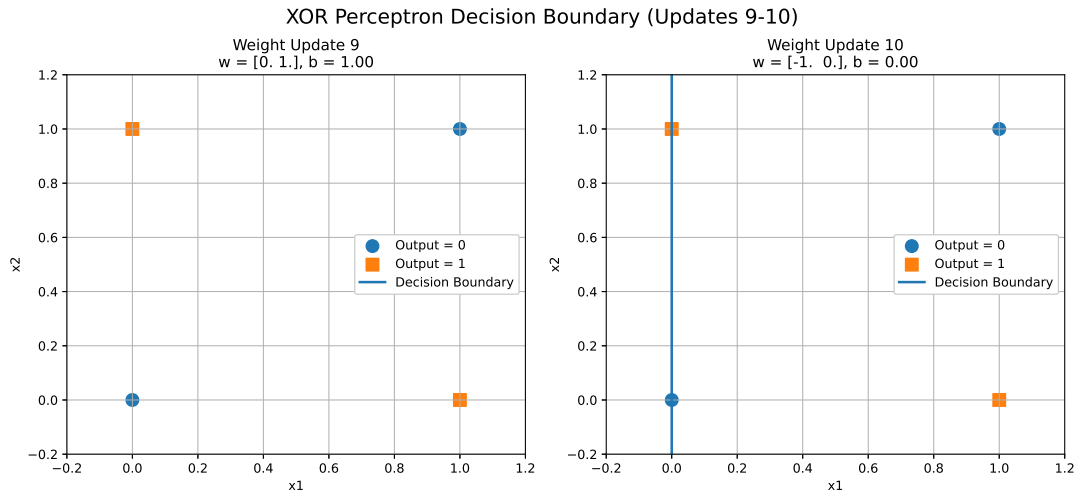


Figure 20: XOR perceptron decision boundaries for weight updates 9–10.

Final Result:

After 10 updates, the final weights were $[-1, 0]$ and the final bias was 0. The final predictions were:

Input	Target	Prediction
(0,0)	0	1
(0,1)	1	1
(1,0)	1	0
(1,1)	0	0

Observation:

The perceptron does not converge since the XOR function cannot be separated linearly. This decision boundary changes constantly while the algorithm tries to rectify errors.

Analysis:

In the XOR problem, the classes are placed at diagonally opposite corners. As a result, a linear separation between the classes is impossible. This results in the perceptron continuously adjusting its weights without ever arriving at the right answer.

3. MLP for XOR Gate

3.1 MLP Architecture

The MLP used for the XOR problem consists of an input layer with two neurons, one hidden layer containing two neurons, and an output layer containing one neuron. Sigmoid activation is used in both the hidden and output layers.

Parameter	Value
Architecture	2-2-1
Hidden Layers	1
Hidden Neurons	2
Hidden Activation	Sigmoid
Output Neurons	1
Output Activation	Sigmoid
Learning Rate	2.0
Maximum Epochs	10,000
Loss Function	Binary Cross-Entropy
Convergence Threshold	0.01
Random Seed	42

3.2 Training Loss

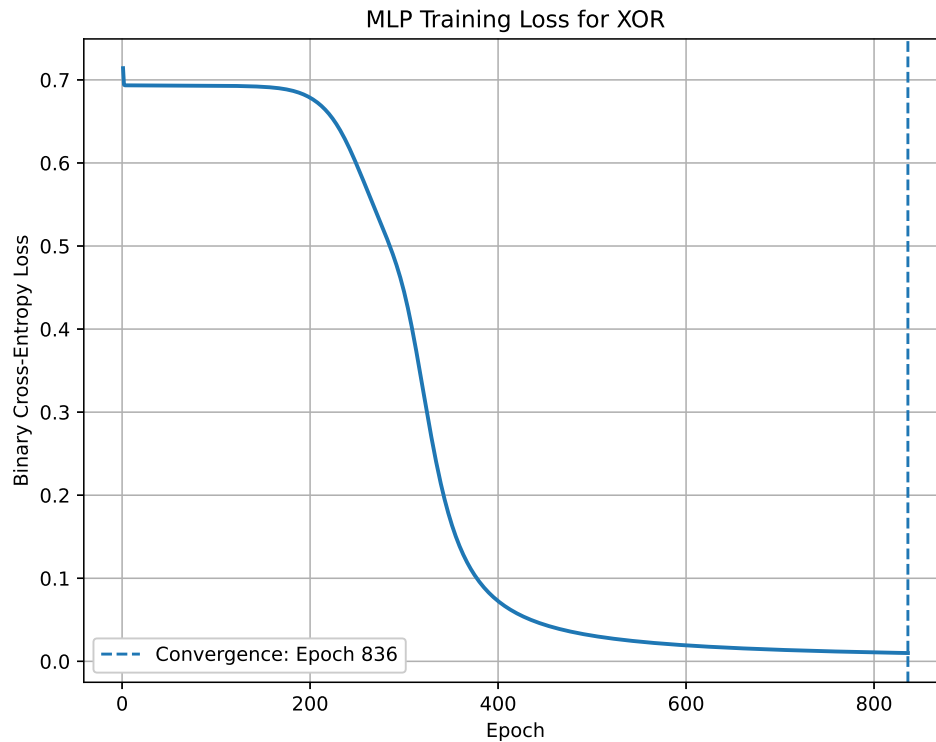


Figure 21: Training loss of the MLP for the XOR gate.

Observation:

The training loss drops from around 0.7141 at epoch 1 to below the convergence criteria of 0.01. The loss decreases sharply after around 300 epochs, which shows that the MLP is learning the nonlinear relationship of XOR.

3.3 Decision Boundary

Observation:

The trained MLP successfully creates a nonlinear decision boundary that can segregate the XOR classes. In contrast to the single-layer perceptron, the hidden layer allows the network to learn the nonlinear relationship in the XOR data set.

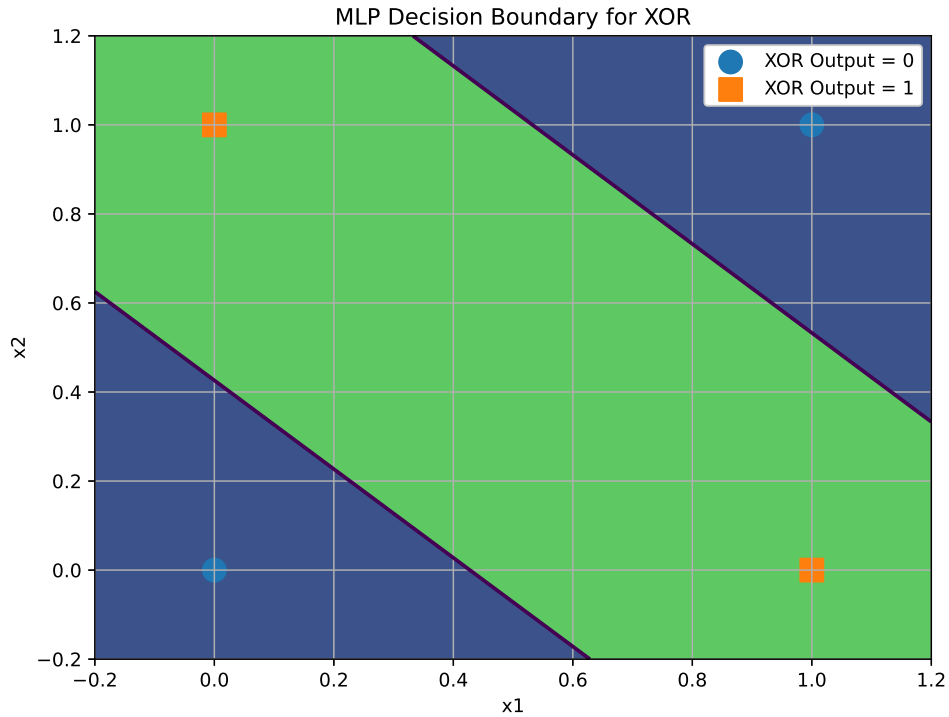


Figure 22: Decision boundary obtained using the trained MLP for the XOR gate.

3.4 Convergence

Convergence Epoch: 836

Final Loss: 0.00999511

Final Accuracy: 100.00%

Observation:

At the 836th epoch, MLP converged with loss less than 0.01 and accuracy being equal to 100%. Final predictions classify all four XOR inputs.

3.5 Final XOR Predictions

Input	Target	Output	Prediction
(0,0)	0	0.012011	0
(0,1)	1	0.990988	1
(1,0)	1	0.990964	1
(1,1)	0	0.009719	0

Observation:

The MLP makes correct predictions for all four XOR data. The outputs are near 0 for the negative data class and near 1 for the positive data class; hence, the classification accuracy is 100%.

Overall Observation

For OR, AND and NOT gates, the perceptron was able to successfully converge because they have linearly separable functions. However, the perceptron was unable to converge on XOR since the classes are not linearly separable. It is the addition of the hidden layer and the nonlinear sigmoid activation that enabled the MLP to be 100% accurate after 836 epochs.

Conclusion

The perceptron learning algorithm was able to learn the logic gates OR, AND, and NOT using iterations in updating weights and biases. Nevertheless, the algorithm was unable to learn the XOR gate since it is not linearly separable. For this reason, an MLP with a 2-2-1 network structure was employed to learn the nonlinear relationship in XOR. The MLP learned at epoch 836 with 0.00999511 as its final loss and reached 100% accuracy.